

Week05 | 实验 | 把客服运营口径写进工程:用 dbt 产出 KPI 包, 并封装安全指标查询工具 v1

本页结构

跑出 Week05 的最小 analytics engineering 闭环	1
这次实验要完成什么	1
参考实验时间	2
本次实验产出	2
先看一张闭环图	2
先看证据文件怎么互相支撑	3
0. 环境前提	3
实验步骤导航	4
1. 启动依赖	5
2. 准备工单数据	5
3. dbt debug	5
4. dbt build	6
5. 生成 docs / lineage evidence	6
6. 校验 metric registry	6
7. 运行受控 KPI 工具正负例	6
8. 验证 Tool API 端点	7
9. 回归检查	8
10. 写实验总结	8
验收不变量	8
排障提示	9
延伸阅读	9
Footnotes	10

跑出 Week05 的最小 analytics engineering 闭环

这次实验不是在项目外做一个独立 dbt demo。

你要围绕当前 OmniSupport Copilot 工程, 把口径从 source 一路推进到 mart、registry、tool contract、runtime guard 和 audit evidence:

让 BI 和 Agent 都能消费同一套受控 KPI。

这次实验要完成什么

本次实验以 [omnisupport-copilot/runbooks/week05/README.md](#) 为真实执行顺序, 覆盖:

1. 启动 PostgreSQL / MinIO / devbox 依赖;
2. 如需要, 生成并导入 Week05 工单数据;
3. `dbt debug` 验证 profiles 和连接;
4. `dbt build --select tag:week05` 构建 sources / staging / intermediate / marts;
5. `dbt docs generate` 生成 docs / manifest / catalog / lineage artifacts;
6. 校验 `analytics/metric_registry_v1.yml`;

7. 运行 `query_support_kpis_v1` 正例和负例；
8. 验证 Tool API endpoint；
9. 跑 Week01—Week05 回归测试；
10. 生成 `reports/week05/week05_lab_execution_summary.md`。

这不是“跑命令打卡”。真正目标是把 source 到 mart 的 transform 跑通，用 tests / docs / lineage 证明口径可信，用 registry 固定可查询指标，用 tool contract 验证 Agent 不能乱查，最后用 evidence 文件把过程交付出来。

参考实验时间

建议按一次完整实验安排：先跑通命令，再整理 evidence、排障记录和实验总结。

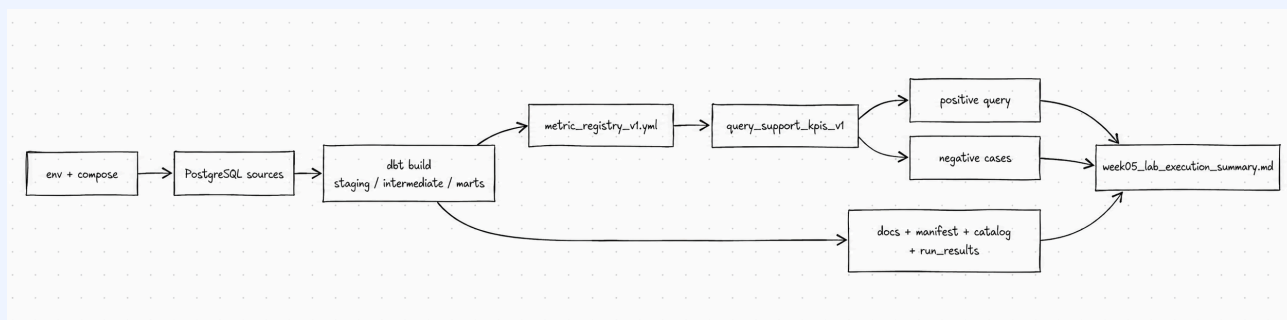
本次实验产出

产出	路径	合格标准
dbt project	<code>analytics/</code>	source、staging、intermediate、marts、tests、docs 结构清楚
KPI mart	<code>analytics/models/marts/support_kpi_mart.sql</code>	能产出 metric rows
safe view	<code>analytics/models/marts/agent_tool_input_view.sql</code>	不暴露 PII、正文和 raw SQL 入口
metric registry	<code>analytics/metric_registry_v1.yml</code>	validator 通过，至少 6 个指标
tool contract	<code>contracts/tools/tools/query_support_kpis_v1.json</code>	schema、拒绝码、审计字段完整
evidence	<code>reports/week05/dbt_build_evidence.md</code>	记录 build、docs、registry、tool、pytest 结果
examples	<code>reports/week05/query_tool_examples.md</code>	正例和负例可复核
summary	<code>reports/week05/week05_lab_execution_summary.md</code>	写清结果、失败处理、下一步

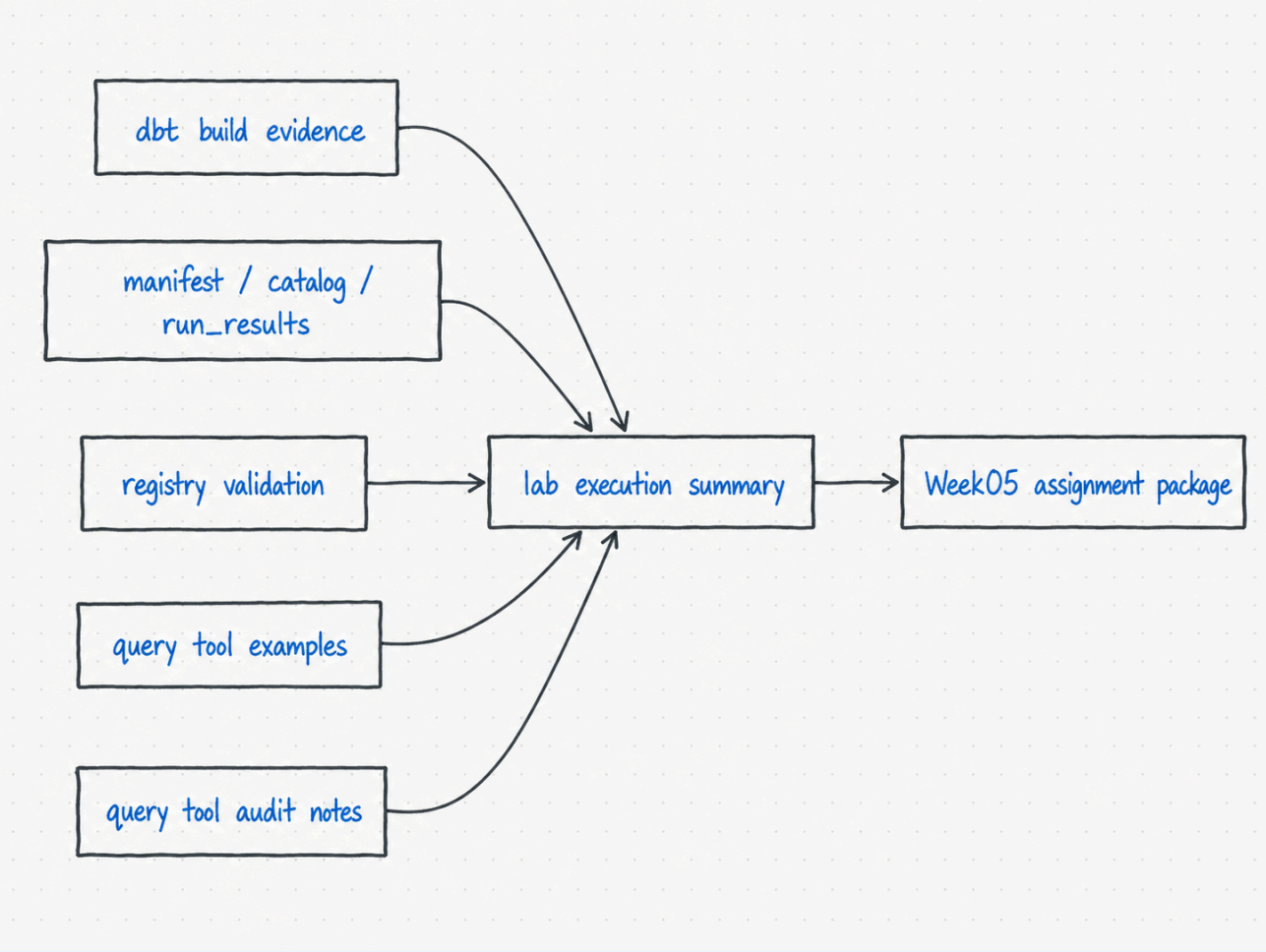
💡 模板入口

实验总结、工具正负例和 delivery summary 可以参考附录的 [Week05 模板库](#)，不要把临时 logs 或 `analytics/target/` 当作正式模板。

先看一张闭环图



先看证据文件怎么互相支撑



实验页验证“能跑通”，作业页整理“能交付”。不要把 `analytics/target/` 或临时日志当作主要提交物；它们是本地证据来源，正式交付应落到 `reports/week05/*.md`。¹

0. 环境前提

本实验默认在 `omnisupport-copilot` 项目仓库根目录执行。

开始前先确认：

- 你没有在课程站点仓库里运行项目命令；
- Docker Desktop / Docker Compose 可用；
- `infra/env/.env.example` 存在；
- 你能看到 `analytics/`、`contracts/tools/tools/query_support_kpis_v1.json`、`services/tool_api/`、`runbooks/week05/README.md`；
- Week03 / Week04 的本地临时输出不需要提交到 main。

前置检查	合格标准	不满足时先做什么
当前目录	是 <code>omnisupport-copilot</code> 项目根目录	不要在课程站点仓库跑项目命令
<code>.env.local</code>	<code>infra/env/.env.local</code> 存在且和 <code>.env.example</code> 对齐	先复制示例并补缺项

¹`analytics/target/` 是本地 dbt 运行输出，适合截图、引用和排障，但通常不作为课程主交付物提交。

前置检查	合格标准	不满足时先做什么
Docker Compose	<code>docker compose</code> 可用	先启动 Docker Desktop
Week05 工程文件	<code>analytics/</code> 、 <code>tool contract</code> 、 <code>runbook</code> 都存在	先拉取项目最新分支
runbook	<code>runbooks/week05/README.md</code> 存在	命令以项目 runbook 为准
仓库边界	课程站点和项目仓库不混用	回到正确目录再执行

! Lab 的第一条规则

课程站点只负责讲义，实验命令在 `omnisupport-copilot` 项目仓库里执行。两个仓库不要混着跑。²

实验步骤导航

步骤	为什么做	期望看到	常见失败先看
1. 启动依赖	只把 Week05 需要的 postgres / minio / init 起起来	服务容器 healthy	Docker、 端口、 <code>.env.local</code>
2. 准备工单数据	确保 source 有数据可 transform	source 表有记录	Week03 ingest state / report
3. <code>dbt debug</code>	验证 profile 和数据库连接	connection passed	<code>DBT_PROFILES_DIR=.</code> 、服务名
4. <code>dbt build</code>	build + tests 一起验收	tag:week05 通过	source、staging、mart grain
5. docs / lineage	生成可解释证据	manifest / catalog / run_results	docs 描述是否缺失
6. registry 校验	确认指标白名单可读	<code>valid=true</code>	YAML 字段、safe view 字段
7. 工具正负例	验证允许和拒绝都可控	正例 allowed, 负例 denial code	registry、role、window
8. API endpoint	验证服务层边界	health 和 POST 正常	端口、service build
9. 回归检查	防止 Week1-4 退化	contract / integration tests 通过	先看失败周次
10. 实验总结	把过程变成交付证据	summary 写清结果和失败处理	缺 evidence 文件

²Docker-first 是为了降低本机环境差异，让 dbt、profile、Postgres、MinIO 和工具依赖在同一套 compose 配置里复现。

1. 启动依赖

```
cp infra/env/.env.example infra/env/.env.local

docker compose --env-file infra/env/.env.local -f infra/docker-compose.yml \
  up -d --build postgres minio minio_init
```

如果你已经有 `.env.local`，不要直接覆盖。先对照 `.env.example` 补缺项。

2. 准备工单数据

如果本地已有 Week03 / Week04 数据，可以直接进入 dbt。否则先生成并导入一批工单数据：

```
docker compose --profile tools --env-file infra/env/.env.local \
  -f infra/docker-compose.yml run --rm devbox \
  python data/synthetic_generators/ticket_simulator.py --count 500 \
    --output data/canonization/tickets/tickets-seed-week05.jsonl

docker compose --profile tools --env-file infra/env/.env.local \
  -f infra/docker-compose.yml run --rm devbox \
  python -m pipelines.ingestion.ticket_ingest \
    --input data/canonization/tickets/tickets-seed-week05.jsonl \
    --batch-id week05-demo
```

期望输出：

- ticket ingest 完成；
- PostgreSQL 中有 `ticket_fact`、`customer_dim`、`ticket_comment_fact` 等 source；
- 失败时优先回到 Week03 ingest state / report 排查。

3. dbt debug

```
docker compose --profile tools --env-file infra/env/.env.local \
  -f infra/docker-compose.yml run --rm devbox \
  bash -lc 'cd analytics && DBT_PROFILES_DIR=. dbt debug'
```

期望输出：

- profiles 能加载；
- adapter 能连接 Docker 网络内的 postgres；
- `analytics/models/sources.yml` 中的 `omni_postgres` source 不需要下游猜表。

常见错误：

错误	优先检查
profile not found	是否在 <code>analytics/</code> 内设置 <code>DBT_PROFILES_DIR=.</code>
could not translate host name	是否在 Docker devbox 内执行，服务名是否为 <code>postgres</code>
password authentication failed	<code>.env.local</code> 和 <code>profiles.yml</code> 是否一致

4. dbt build

```
docker compose --profile tools --env-file infra/env/.env.local \
  -f infra/docker-compose.yml run --rm devbox \
  bash -lc 'cd analytics && DBT_PROFILES_DIR=. dbt build --select tag:week05'
```

期望输出：

- stg_*、int_*、support_case_mart、support_kpi_mart、agent_tool_input_view 构建成功；
- dbt tests 全部通过；
- no_pii_columns_in_agent_tool_input_view 通过；
- 当前项目证据为 37/37 通过，support_case_mart=50，support_kpi_mart=300。

失败后不要先改 SQL。先判断：

- source 表是否存在；
- source 字段是否和 sources.yml 对齐；
- staging 是否把枚举和时间字段规范化；
- mart 粒度是否导致重复；
- safe view 是否暴露了不该暴露的字段。

5. 生成 docs / lineage evidence

```
docker compose --profile tools --env-file infra/env/.env.local \
  -f infra/docker-compose.yml run --rm devbox \
  bash -lc 'cd analytics && DBT_PROFILES_DIR=. dbt docs generate'
```

你要检查的不是页面好不好看，而是：

- target/manifest.json 是否生成；
- target/catalog.json 是否生成；
- 关键 model 是否有描述；
- lineage 是否能从 source 追到 marts；
- agent_tool_input_view 是否只暴露 safe columns。

analytics/target/ 是本地 runtime 输出，通常不提交到 main。

6. 校验 metric registry

```
docker compose --profile tools --env-file infra/env/.env.local \
  -f infra/docker-compose.yml run --rm devbox \
  python analytics/scripts/validate_metric_registry.py --json
```

期望输出：

- valid: true
- metric_count >= 6
- safe view 字段包含 metric_date、metric_name、metric_value 和允许维度；
- max_window_days 没有被无意放大。

7. 运行受控 KPI 工具正负例

正向调用：

```
docker compose --profile tools --env-file infra/env/.env.local \
-f infra/docker-compose.yml run --rm devbox \
bash -lc 'PYTHONPATH=services/tool_api python -m app.kpi_query --example valid'
```

未知指标被拒绝：

```
docker compose --profile tools --env-file infra/env/.env.local \
-f infra/docker-compose.yml run --rm devbox \
bash -lc 'PYTHONPATH=services/tool_api python -m app.kpi_query --example bad_metric || true'
```

角色无权限被拒绝：

```
docker compose --profile tools --env-file infra/env/.env.local \
-f infra/docker-compose.yml run --rm devbox \
bash -lc 'PYTHONPATH=services/tool_api python -m app.kpi_query --example bad_role || true'
```

验收点：

用例	期望
valid	allowed=true, 有 rows 和 audit
bad_metric	allowed=false, denial_code=METRIC_DENIED
bad_role	allowed=false, denial_code=ROLE_DENIED

8. 验证 Tool API 端点

```
docker compose --env-file infra/env/.env.local \
-f infra/docker-compose.yml up -d --build tool_api
```

```
curl -s http://localhost:8001/health
```

```
curl -s -X POST http://localhost:8001/api/v1/tools/query_support_kpis \
-H 'Content-Type: application/json' \
-H 'X-Actor-ID: instructor-local' \
-d '{
  "actor_role": "instructor",
  "metrics": ["ticket_count"],
  "date_from": "2026-04-01",
  "date_to": "2026-04-30",
  "dimensions": ["product_line", "priority"],
  "limit": 20
}'
```

期望输出：

- /health 正常；
- POST 返回 allowed=true；
- audit.actor_id 能带上 X-Actor-ID；
- row count 有上限，不返回无限结果。

9. 回归检查

```
docker compose --profile tools --env-file infra/env/.env.local \
-f infra/docker-compose.yml run --rm devbox \
pytest tests/contract/test_json_schemas.py \
tests/contract/test_week02_gate.py \
tests/contract/test_week05_metric_contracts.py -q
```

```
docker compose --profile tools --env-file infra/env/.env.local \
-f infra/docker-compose.yml run --rm devbox \
pytest tests/integration/test_ingest_state.py \
tests/integration/test_replay_backfill_dry_run.py \
tests/integration/test_week4_lakehouse_smoke.py \
tests/integration/test_week05_metric_registry.py \
tests/integration/test_week05_kpi_query_tool.py -q
```

通过标准：

- Week01 / Week02 contract gate 不退化；
- Week03 ingest state / replay / backfill dry-run 不退化；
- Week04 lakehouse smoke 不退化；
- Week05 registry 和 KPI 工具测试通过。³

10. 写实验总结

最后补齐 `reports/week05/week05_lab_execution_summary.md`，至少回答：

1. 哪些 source 被 dbt 接住；
2. 哪些 models 被 build；
3. 哪些 tests 保护了核心口径；
4. docs / lineage artifacts 是否生成；
5. registry 中开放了哪些指标；
6. KPI 工具正例和负例结果是什么；
7. 哪些本地 target / logs 不提交；
8. 下一步作业要整理哪些正式交付物。

验收不变量

检查点	必须满足	失败时先看
dbt debug	profile 和 Postgres 连接通过	<code>DBT_PROFILES_DIR=.</code> 、 <code>devbox</code> 、 <code>.env.local</code>
dbt build --select tag:week05	models 和 tests 通过	source 表、字段、mart grain
docs artifacts	<code>manifest.json</code> 、 <code>catalog.json</code> 、 <code>run_results.json</code> 生成	dbt docs generate 输出
registry	<code>valid=true</code> 且 <code>metric_count >= 6</code>	<code>analytics/metric_registry_v1.yml</code>
正例	<code>allowed=true</code> ，有 rows 和 audit	日期窗口、seed 数据、role

³Week05 改的是 analytics 和 tool-safe 查询，但它依赖 Week01-4 的合同、采集状态和湖仓基线。回归测试能防止本周改动把前面周次的工程链路破坏掉。

检查点	必须满足	失败时先看
负例	返回对应 <code>denial_code</code>	<code>metric / role / dimension / window guard</code>
回归	Week01–Week05 合同和集成测试不退化	先定位失败所属周次
summary	<code>week05_lab_execution_summary.md</code> 存在	是否记录 <code>build</code> 、 <code>docs</code> 、 <code>registry</code> 、 <code>tool</code> 、 <code>pytest</code>

💡 录屏建议

如果要做 6–8 分钟演示，录 `dbt debug` → `dbt build` → `registry validator` → KPI 工具正例 → KPI 工具负例。重点讲“为什么负例通过才说明工具边界可信”。⁴

排障提示

问题	优先检查
<code>profile not found</code>	是否在 <code>analytics/</code> 内运行，是否设置 <code>DBT_PROFILES_DIR=.</code>
Postgres connection refused	postgres 容器是否启动，devbox 是否在 Docker 网络内
source table not found	Week03/Week04 数据是否导入， <code>sources.yml</code> 表名是否真实
dbt test failed	先看失败字段和 grain，不要直接放宽测试
registry invalid	metric name、allowed dimensions、safe view 字段是否对齐
<code>PYTHONPATH=services/tool_api</code> 模块找不到	是否在项目根目录执行，路径是否打错
API 端口不可访问	<code>tool_api</code> 是否 build / up，端口 8001 是否被占用
时间窗口被拒绝	先确认是否超过 <code>max_window_days: 31</code> ，这通常不是 bug
正例没数据	先检查日期窗口和 seed 数据，而不是放大工具权限

延伸阅读

主题	推荐资料	为什么读
dbt build	dbt Docs: build command	理解 build 为什么适合实验验收
dbt data tests	dbt Docs: Data tests	理解核心口径的自动检查
dbt docs generate	dbt Docs: docs generate	理解 docs / catalog 证据来自哪里

⁴安全指标工具的核心不是“能查成功”，而是“不该查的能稳定拒绝”。负例验证比正例更能证明边界存在。

主题	推荐资料	为什么读
dbt artifacts	dbt Docs: Artifacts	理解 manifest / catalog / run_results 的作用
OpenAI Function Calling	OpenAI Docs: Function Calling	理解工具调用为什么需要契约
OpenAI Structured Outputs	OpenAI Docs: Structured Outputs	理解结构化工具输出和 schema 的关系

Footnotes